

```

root@Tridev-Reddy: /opt/zeek/bin
root@Tridev-Reddy:/opt/zeek/bin# ls /opt/zeek/logs/current
loaded_scripts.log packet_filter.log stats.log stderr.log stdout.log
root@Tridev-Reddy:/opt/zeek/bin# wget https://google.com/
--2022-01-26 13:51:53-- https://google.com/
Resolving google.com (google.com)... 142.250.77.78, 2404:6800:4009:829::200e
Connecting to google.com (google.com)|142.250.77.78|:443... connected.
HTTP request sent, awaiting response... 301 Moved Permanently
Location: https://www.google.com/ [following]
--2022-01-26 13:51:54-- https://www.google.com/
Resolving www.google.com (www.google.com)... 172.217.166.36, 2404:6800:4009:813::2004
Connecting to www.google.com (www.google.com)|172.217.166.36|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: unspecified [text/html]
Saving to: 'index.html.1'

index.html.1          [ <==>          ] 17.70K  64.4KB/s   in 0.3s

2022-01-26 13:51:56 (64.4 KB/s) - 'index.html.1' saved [18120]

root@Tridev-Reddy:/opt/zeek/bin# ls /opt/zeek/logs/current
capture_loss.log  known_hosts.log  packet_filter.log  stderr.log
conn.log          known_services.log  ssl.log            stdout.log
dns.log           loaded_scripts.log  stats.log

```

Figure 5: Log files

```

tridev@Tridev-Reddy:/opt/zeek/bin
# sudo ./zeekctl deploy
checking configurations ...
installing ...
removing old policies in /opt/zeek/spool/installed-scripts-do-not-touch/site ...
removing old policies in /opt/zeek/spool/installed-scripts-do-not-touch/auto ...
creating policy directories ...
installing site policies ...
generating standalone-layout.zeek ...
generating local-networks.zeek ...
generating zeekctl-config.zeek ...
generating zeekctl-config.sh ...
stopping ...
stopping zeek ...
starting ...
starting zeek ...

```

Figure 6: Analysing *conn.log*

```

proccoder101@proccoder101-Inspiron-3593: /usr/local/zeek/log$ ./zeek -r
File Edit View Search Terminal Help
proccoder101@proccoder101-Inspiron-3593: /usr/local/zeek/log$ zeek-cut < conn.log ts id.orig_h id.resp_h proto
1643177770.397849 192.168.29.35 224.0.0.251 udp
1643177781.941311 2405:201:800b:a01d:6d8c:5930:eacb:cd85 2405:201:800b:a01d:c0a8:1d01 udp
1643177781.943860 2405:201:800b:a01d:6d8c:5930:eacb:cd85 2405:201:800b:a01d:c0a8:1d01 udp
1643177781.946362 2405:201:800b:a01d:6d8c:5930:eacb:cd85 2405:201:800b:a01d:c0a8:1d01 udp
1643177806.868729 2405:201:800b:a01d:6d8c:5930:eacb:cd85 2404:6800:4009:82c::2004 tcp
1643177807.994564 2405:201:800b:a01d:6d8c:5930:eacb:cd85 2404:6800:4009:82d::2003 tcp
1643177808.089488 2405:201:800b:a01d:6d8c:5930:eacb:cd85 2404:6800:4009:809::200e tcp
1643177808.155352 2405:201:800b:a01d:6d8c:5930:eacb:cd85 2404:6800:4009:801::2001 tcp
1643177808.334963 192.168.29.146 142.250.206.142 tcp
1643177808.083978 2405:201:800b:a01d:6d8c:5930:eacb:cd85 2404:6800:4009:81f::200e tcp
1643177808.497713 2405:201:800b:a01d:6d8c:5930:eacb:cd85 2404:6800:4009:831::2003 tcp
1643177808.559860 2405:201:800b:a01d:6d8c:5930:eacb:cd85 2404:6800:4009:831::2003 tcp
1643177809.234191 2405:201:800b:a01d:6d8c:5930:eacb:cd85 2404:6800:4009:81f::200e tcp
1643177810.245916 2405:201:800b:a01d:6d8c:5930:eacb:cd85 2404:6800:4009:81a::2002 tcp
1643177810.458475 2405:201:800b:a01d:6d8c:5930:eacb:cd85 2404:6800:4009:827::2002 tcp
1643177810.048952 2405:201:800b:a01d:6d8c:5930:eacb:cd85 2404:6800:4009:824::2002 tcp
1643177812.375849 2405:201:800b:a01d:6d8c:5930:eacb:cd85 2404:6800:4002:81f::200e tcp
1643177807.993438 2405:201:800b:a01d:6d8c:5930:eacb:cd85 2404:6800:4009:801::2001 tcp
1643177806.854372 2405:201:800b:a01d:6d8c:5930:eacb:cd85 2405:201:800b:a01d:c0a8:1d01 udp
1643177806.854552 2405:201:800b:a01d:6d8c:5930:eacb:cd85 2405:201:800b:a01d:c0a8:1d01 udp

```

Figure 7: Extracting certain fields with *zeek-cut*

Zeek configurations. For doing that, execute the following command in the same directory:

```
sudo ./zeekctl deploy
```

On successful deployment, the output on the terminal will be as shown in Figure 4.

Step 9: Next, we need to check the status of Zeek. The output of the command given below will show the

status with PID:

```
sudo ./zeekctl status
```

Zeek in action

Here's a quick demo on how to sniff data and analyse logs using Zeek. Start Zeek using the command:

```
./zeekctl start
```

To demonstrate and generate some

traffic, crawl through the Web page and grab *index.html* from *Google.com* and also ping *Google*:

```
ping 8.8.8.8
```

Figure 5 shows that there are a few more files in the *logs/current* directory. Once Zeek is stopped, these files will be stored in a separate new folder with the current date. After stopping the *zeek* service, the files in the current directory will be deleted but you do not need to worry as they will be stored in a new folder.

Let us see a file named *conn.log* and analyse the data.

The complete data that has been sniffed can be seen clearly in Figure 6 — the IP from which the traffic is generated, to which IP, UID, time, etc. This is how we use Zeek to analyse network traffic.

Another cool feature of Zeek is its capability to import any *pcap* file and analyse the traffic from that file using the argument *-r*:

```
./zeek -r
```

So, we have seen how to get started with Zeek. You can try out other features of Zeek as you start using it.

Faster log analysis with *zeek-cut* and UNIX tools

As evident from Figure 6, with the high amount of network transactions, the log files become over cluttered and difficult to read. In such scenarios, we may want to filter out only certain columns and/or certain rows. The utility *zeek-cut*, provided as a part of the Zeek package itself, can be of great help here, as it allows us to pick up certain columns of data.

The syntax is simple:

```
./zeek-cut < log_file_name columns_to_be_extracted
```

For example, we may try to pick out only the time stamp, the origin address, the destination address and the protocol